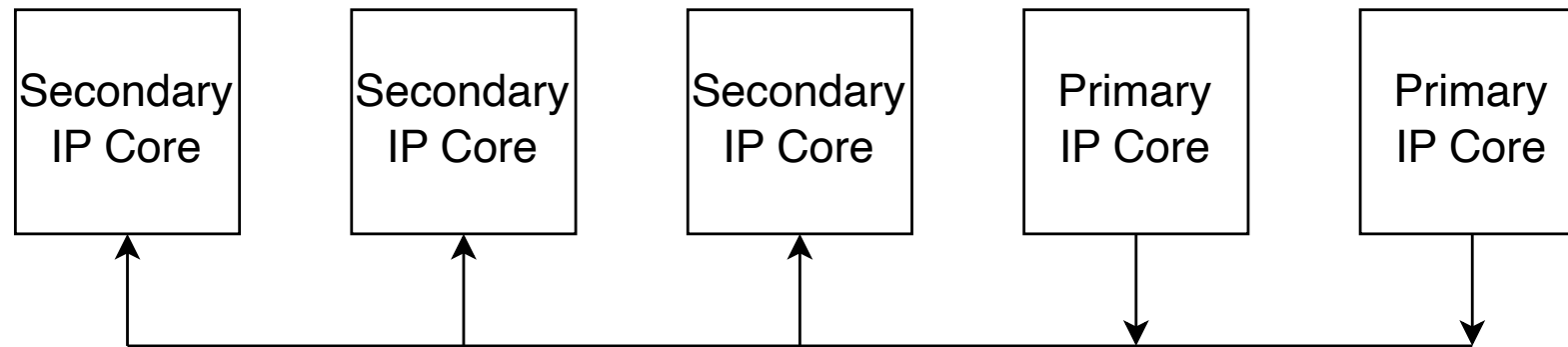


Lecture 27

Buses

Conventional Bus Structure



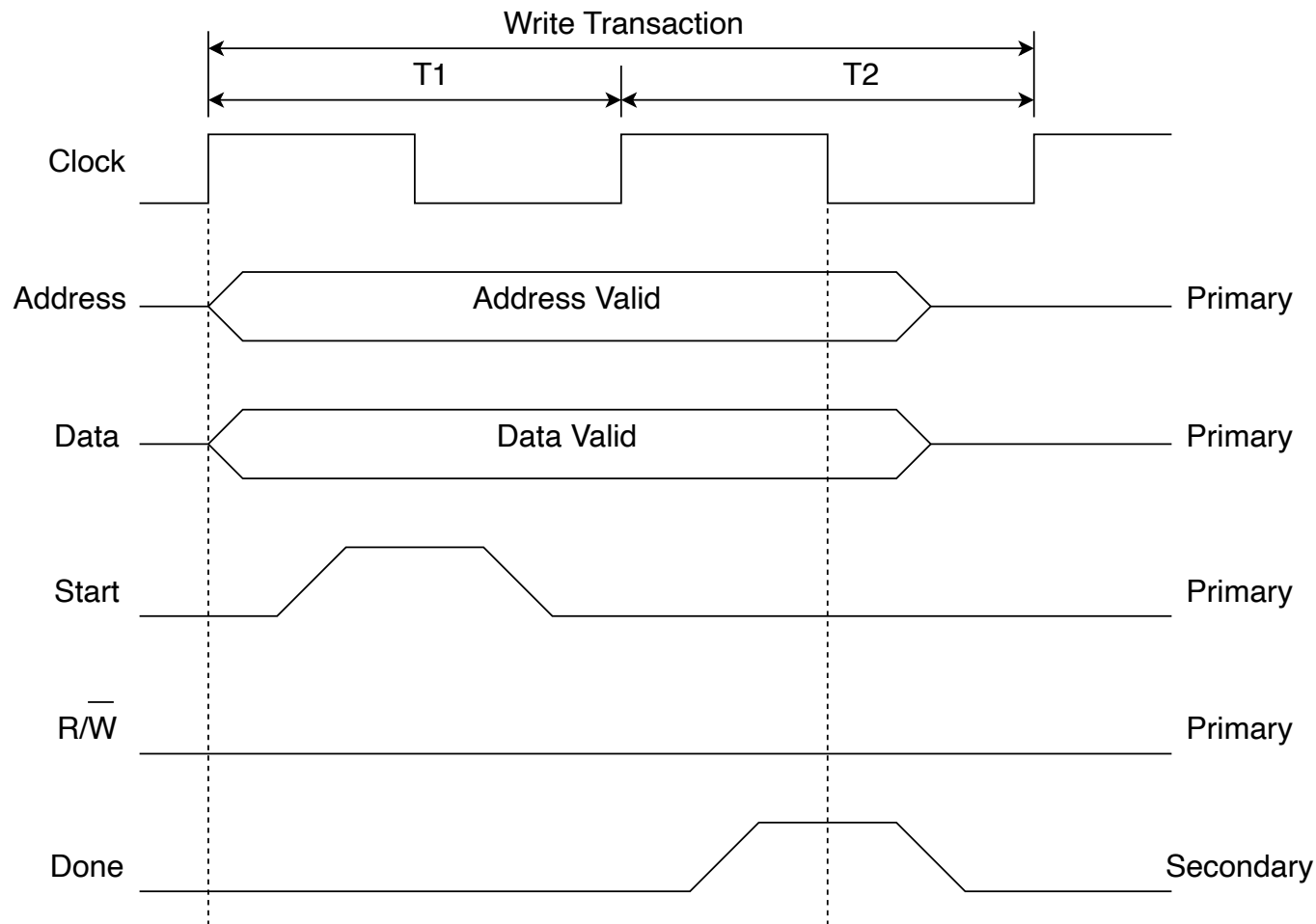
- Bus consists of shared data, address, and control signals (3 buses?)
- Only 1 Primary can write to the bus (e.g. CPU)
- Note. These slides use the terminology "Primary"/"Secondary"
- You will also see the terms "Master"/"Slave"
 - No longer considered appropriate
 - Not a good description of the protocol

A Simple Protocol

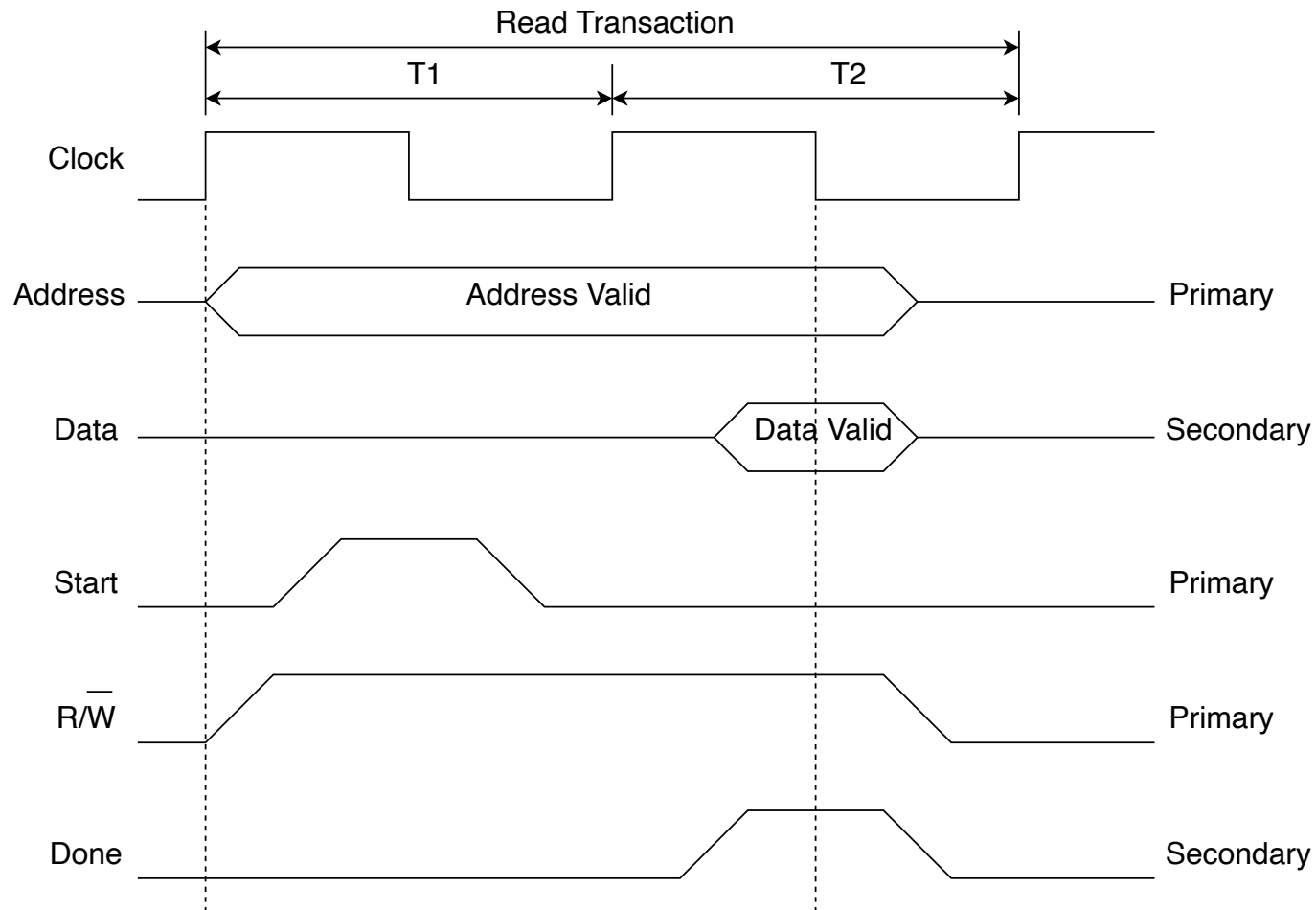
Signal	Set by
Start	Primary
Data	Primary/Secondary
Address	Primary
R/~W	Primary
Done	Secondary
Clock	Bus

- Assume 2 cycles for each transaction
- Transactions start on rising edge of T_1
- Transactions complete on falling edge of T_2

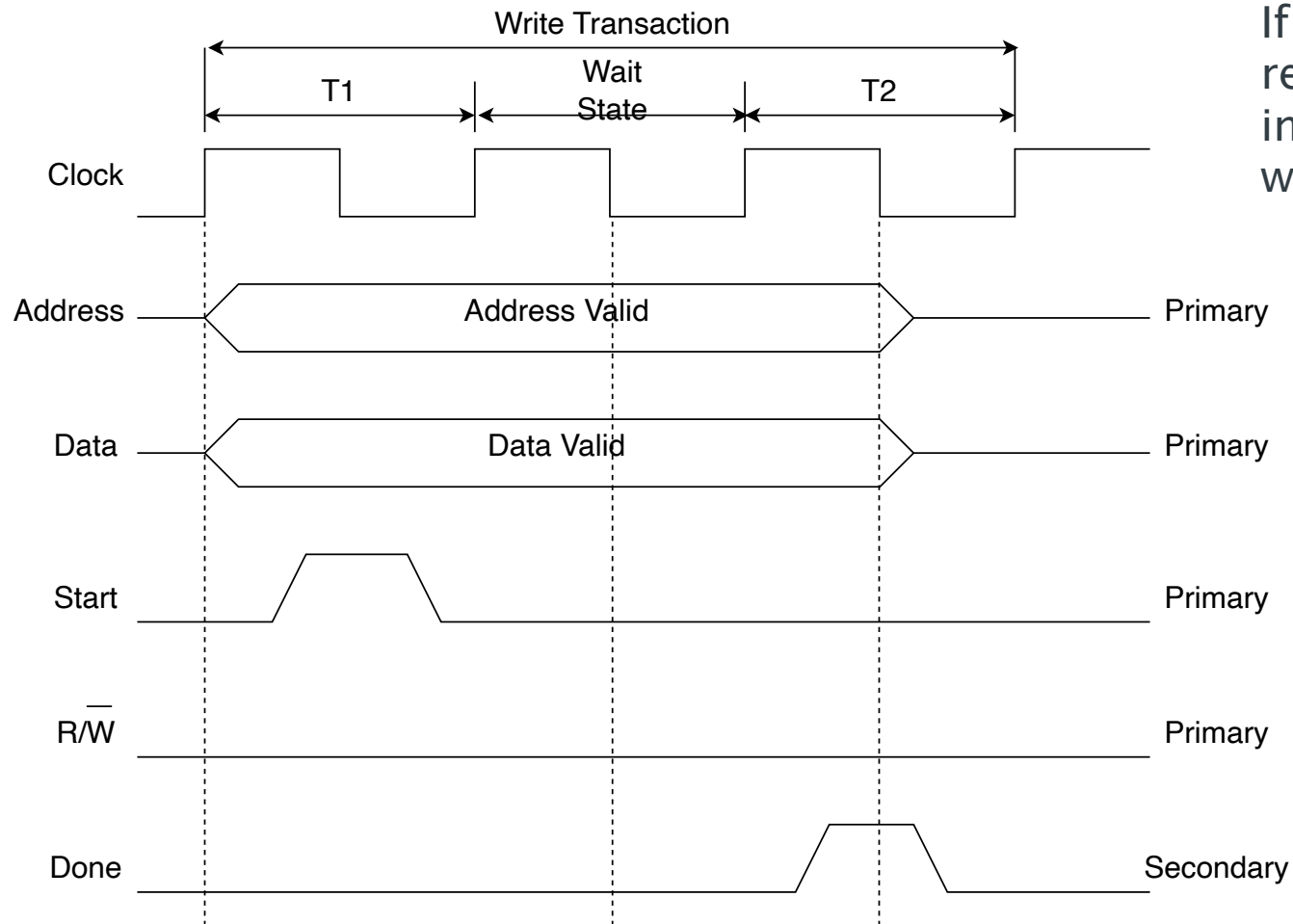
Write Transaction



Read Transaction



Wait state



If the secondary can't respond quickly enough, insert 1 or more wait states.

Bus Arbitration

- In a system (on chip), the CPU might be the only primary
- *Direct-Memory Access* (DMA) allows data to be written to, or read from memory, without going through the CPU
- We now have 2 primaries – which one has priority?
- Bus arbiter decides
 - Arbitration can defined by the system designer
 - Priority
 - "Round-robin"
 - etc

Bus Standards

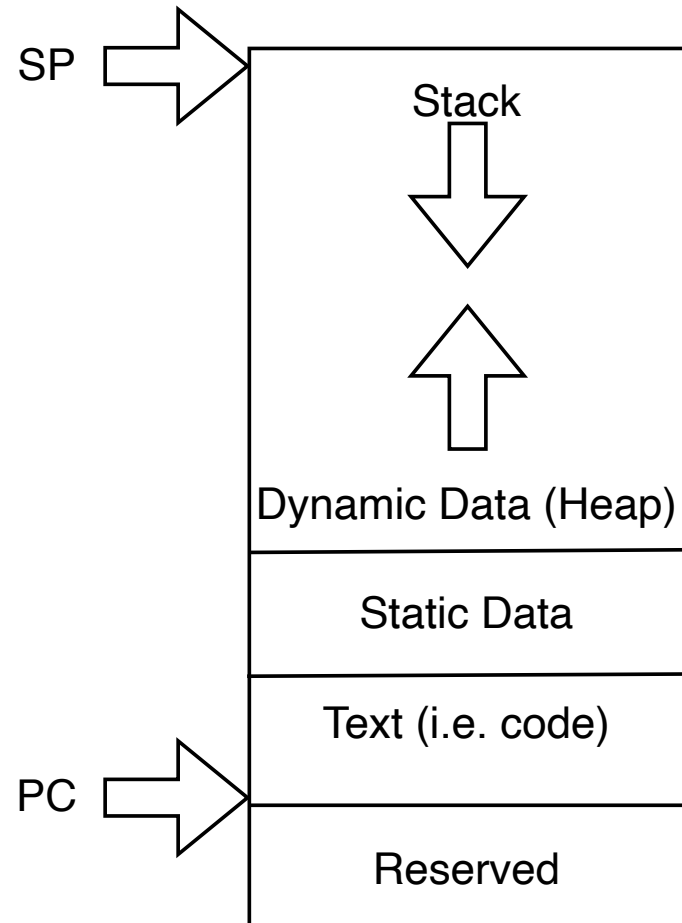
- Various bus standards exist
 - Wishbone
 - Open standard (opencores.org)
 - 128 pages
 - Open Core Protocol (OCP)
 - Accellera. Open standard
 - 500 pages
 - Advanced Microcontroller Bus Architecture (AMBA)
 - Arm. Open standard(s)
 - Different versions (AHB, AXI, etc.)

Bus IP

- (Jargon: IP stands for Intellectual Property. In this context, refers to packaged building blocks, typically written in SystemVerilog)
- Bus interfaces are usually available as IP blocks, either as library cells in Vivado, or as open source code from, e.g. opencores.org
 - Don't write them yourself
- Bus IP also includes interfaces between different standards

Input/Output

Memory-Mapped I/O



- RISC-V, and other RISC architectures, uses memory mapped I/O
- I/O is mapped to static data region
- See Lab 3 for an example

Port-Mapped I/O

- The alternative is to have special "ports" (outside the address space) that are written to/read from by special instructions
- May be limited number of ports
- Added complexity
- But such a processor could write to/read from I/O in the memory address space

Interrupts

- I/O is generally very slow compared with the processor
- In general, we don't want the processor to wait until new data arrives – do something useful while it's waiting
- *Polling* – periodically check for new data
- *Interrupt* – An exception is raised and the CPU responds
- RISC-V, and other processors, don't really distinguish between exceptions (internal) and interrupts (external) at the software level
- Difficulty comes with multiple interrupts at the same time

Hardware Support for Interrupts

- Choices about hardware structures:
 - Minimal logic – software polling
 - Check each device in turn. No priority
 - Daisy Chaining
 - Hardwire the priority. Devices nearer the CPU have higher priority and block devices further away
 - Programmable Interrupt Controller
 - Dynamically mask interrupts